Yamux와 Mplex 프로토콜 비교

개요

이 문서는 Rust libp2p에서 지원하는 두 가지 주요 스트림 멀티플렉싱 프로토콜인 Yamux와 Mplex의 상세한 비교 분석을 제공합니다. 두 프로토콜 모두 단일 네트워크 연결 위에서 여러 논리적 스트림을 관리하는 기능을 제공하지만, 설계 철학과 구현 방식에서 중요한 차이점을 보입니다.

기본 정보

Yamux (Yet Another Multiplexer)

- **개발자**: HashiCorp
- **프로토콜 버전**: `/yamux/1.0.0`
- **라이브러리**: `libp2p-yamux`
- **기반 구현**: `yamux` crate (v0.12.1, v0.13.3)
- **설계 목표**: 고성능, 흐름 제어, 백프레셔 지원

Mplex (Multiplexer)

- **개발자**: libp2p 커뮤니티
- **프로토콜 버전**: `/mplex/6.7.0`
- **라이브러리**: `libp2p-mplex`
- **기반 구현**: 자체 구현
- **설계 목표**: 단순성, 경량화

아키텍처 비교

Yamux 아키텍처

```
pub struct Muxer<C> {
connection: Either<yamux012::Connection<C>, yamux013::Connection<C>>,
inbound_stream_buffer: VecDeque<Stream>,
inbound_stream_waker: Option<Waker>,
}
```

특징:

- 외부 `yamux` crate 기반 구현
- 두 버전(0.12, 0.13) 동시 지원
- 인바운드 스트림 버퍼링 (최대 256개)
- 정교한 흐름 제어 메커니즘

Mplex 아키텍처

```
pub struct Multiplex<C> {
io: Arc<Mutex<io::Multiplexed<C>>>,
}
```

```
pub(crate) struct Multiplexed<C> {
id: ConnectionId,
status: Status,
```

```
io: Fuse<Framed<C, Codec>>,

config: Config,

open_buffer: VecDeque<LocalStreamId>,

pending_frames: VecDeque<Frame<LocalStreamId>>,

substreams: IntMap<LocalStreamId, SubstreamState>,

// ... 기타 필드들

}
```

특징:

- 완전 자체 구현
- 프레임 기반 프로토콜
- 공유 상태 관리 (Arc)
- 단순한 스트림 관리

프로토콜 세부사항

프레임 구조

Yamux

- **헤더**: 12바이트 고정 헤더
- **필드**: Version, Type, Flags, Stream ID, Length
- **타입**: Data, Window Update, Ping, Go Away
- **흐름 제어**: 윈도우 기반 크레딧 시스템

Mplex

- **헤더**: 가변 길이 헤더 (varint 인코딩)
- **필드**: Stream ID + Flag (3비트), Length
- **타입**: Open, Data, Close, Reset
- **흐름 제어**: 기본적인 버퍼 관리

스트림 식별

Yamux

```
// 스트림 ID는 연결 내에서 고유
// 클라이언트: 홀수 ID, 서버: 짝수 ID
```

Mplex

```
#[derive(Copy, Clone, Eq, Debug)]
pub(crate) struct LocalStreamId {
num: u64,
role: Endpoint, // Dialer 또는 Listener
}

// 프레임 헤더에서 역할 정보 인코딩
// 0: Open (Dialer), 1: Data (Listener), 2: Data (Dialer),
// 3: Close (Listener), 4: Close (Dialer),
// 5: Reset (Listener), 6: Reset (Dialer)
```

설정 및 구성

Yamux 설정

```rust
#[derive(Debug, Clone)]
pub struct Config(Either<Config012, Config013>);

impl Config {
// 최대 동시 스트림 수 설정
pub fn set_max_num_streams(&mut self, num_streams: usize) -> &mut Self;
// 윈도우 업데이트 모드 설정 (deprecated)
pub fn set_window_update_mode(&mut self, mode: WindowUpdateMode) -> &mut Self;
// 수신 윈도우 크기 설정 (deprecated)
pub fn set_receive_window_size(&mut self, num_bytes: u32) -> &mut Self;
}
```

Mplex 설정

```rust
#[derive(Debug, Clone)]
pub struct Config {
max_substreams: usize, // 최대 서브스트림 수 (기본값: 128)
max_buffer_len: usize, // 스트림당 최대 버퍼 프레임 수 (기본값: 32)
max_buffer_behaviour: MaxBufferBehaviour, // 버퍼 오버플로우 동작
split_send_size: usize, // 전송 프레임 최대 크기 (기본값: 8KB)
```

```
protocol_name: &'static str, // 프로토콜 이름
}

#[derive(Debug, Copy, Clone, PartialEq, Eq)]
pub enum MaxBufferBehaviour {
ResetStream, // 버퍼 오버플로우 시 스트림 리셋
Block, // 버퍼 오버플로우 시 읽기 차단 (기본값)
}
```

성능 특성

메모리 사용량

Yamux

- **장점**:
- 외부 최적화된 구현 활용
- 효율적인 윈도우 기반 흐름 제어
- 스트림별 독립적인 백프레셔
- **단점**:
- 더 복잡한 메모리 구조
- 두 버전 지원으로 인한 오버헤드

Mplex

- **장점**:

- 단순한 메모리 구조
- 직접적인 프레임 관리
- 낮은 메타데이터 오버헤드
- **단점**:
- Arc 사용으로 인한 동기화 비용
- 제한적인 흐름 제어

처리량 및 지연시간

Yamux

- **처리량**: 높음 (효율적인 윈도우 관리)
- **지연시간**: 중간 (흐름 제어 오버헤드)
- **확장성**: 우수 (정교한 백프레셔)

Mplex

- **처리량**: 중간 (단순한 프레임 처리)
- **지연시간**: 낮음 (최소한의 오버헤드)
- **확장성**: 제한적 (기본적인 버퍼 관리)

흐름 제어 메커니즘

Yamux 흐름 제어

```
pub struct WindowUpdateMode(yamux012::WindowUpdateMode);
```

```
impl WindowUpdateMode {
```

```
// 데이터 읽기 시 윈도우 업데이트 (권장)
pub fn on_read() -> Self;
// 데이터 수신 시 윈도우 업데이트 (deprecated)
pub fn on_receive() -> Self;
}
```

특징:

- 스트림별 독립적인 수신 윈도우
- 크레딧 기반 흐름 제어
- 백프레셔 전파 지원
- 데드락 방지 메커니즘

Mplex 흐름 제어

```
pub enum MaxBufferBehaviour {
ResetStream, // 오버플로우 시 스트림 종료
Block, // 오버플로우 시 전체 연결 차단
}
```

특징:

- 프레임 수 기반 버퍼 제한
- 연결 레벨 흐름 제어
- 단순한 백프레셔 메커니즘
- 제한적인 데드락 방지

오류 처리

Yamux 오류 처리

```
#[derive(Debug, Error)]

#[error(transparent)]

pub struct Error(Either<yamux012::ConnectionError, yamux013::ConnectionError>);
```

특징:

- 상세한 오류 분류
- 연결 레벨 오류 전파
- 스트림별 독립적인 오류 처리
- 복구 가능한 오류 구분

Mplex 오류 처리

```
enum Status {

Open,

Closed,

Err(io::Error),

}
```

특징:

- 단순한 상태 관리
- 연결 레벨 오류 처리

- 제한적인 오류 복구
- 빠른 실패 메커니즘

사용 사례 및 권장사항

Yamux 권장 사용 사례

1. **고처리량 애플리케이션**

- 대용량 데이터 전송
- 스트리밍 서비스
- 파일 공유 시스템

2. **복잡한 흐름 제어가 필요한 경우**

- 다양한 우선순위의 스트림
- 백프레셔가 중요한 애플리케이션
- 네트워크 조건이 불안정한 환경

3. **장기 실행 연결**

- 지속적인 P2P 연결
- 서버 간 통신
- 실시간 협업 도구

Mplex 권장 사용 사례

1. **경량 애플리케이션**

- 간단한 메시징
- 제어 프로토콜
- IoT 디바이스 통신

2. **지연시간이 중요한 경우**

- 실시간 게임
- 음성/영상 통화
- 인터랙티브 애플리케이션

3. **리소스 제약 환경**

- 임베디드 시스템
- 모바일 애플리케이션
- 배터리 효율성이 중요한 경우

코드 예제

Yamux 사용 예제

```
use libp2p_yamux as yamux;


// 기본 설정
let config = yamux::Config::default();


// 커스텀 설정
let mut config = yamux::Config::default();
config.set_max_num_streams(1000);


// 연결 업그레이드
let transport = tcp::tokio::Transport::new(tcp::Config::default())
```

```
.upgrade(Version::V1Lazy)
.authenticate(noise::Config::new(&keypair)?)
.multiplex(config);
```

Mplex 사용 예제

```
use libp2p_mplex as mplex;

// 기본 설정
let config = mplex::Config::new();

// 커스텀 설정
let mut config = mplex::Config::new()
.set_max_num_streams(256)
.set_max_buffer_size(64)
.set_max_buffer_behaviour(mplex::MaxBufferBehaviour::Block)
.set_split_send_size(16 * 1024);

// 연결 업그레이드
let transport = tcp::tokio::Transport::new(tcp::Config::default())
.upgrade(Version::V1Lazy)
.authenticate(noise::Config::new(&keypair)?)
```

```
.multiplex(config);
```

호환성 및 상호 운용성

프로토콜 네고시에이션

```
// Yamux
impl UpgradeInfo for Config {
type Info = &'static str;
fn protocol_info(&self) -> Self::InfoIter {
iter::once("/yamux/1.0.0")
}
}

// Mplex
impl UpgradeInfo for Config {
type Info = &'static str;
fn protocol_info(&self) -> Self::InfoIter {
iter::once("/mplex/6.7.0") // 또는 커스텀 프로토콜 이름
}
}
```

다른 libp2p 구현과의 호환성

- **Go libp2p**: 두 프로토콜 모두 지원
- **JavaScript libp2p**: 두 프로토콜 모두 지원
- **Python libp2p**: 제한적 지원
- **C++ libp2p**: Yamux 주로 지원

마이그레이션 가이드

Mplex에서 Yamux로 마이그레이션

1. **의존성 변경**

```
# Cargo.toml
[dependencies]
# libp2p-mplex = "0.43" # 제거
libp2p-yamux = "0.47" # 추가
```

2. **설정 변경**

```
// 이전 (Mplex)
let config = mplex::Config::new()
.set_max_num_streams(128);


// 이후 (Yamux)
let mut config = yamux::Config::default();
config.set_max_num_streams(128);
```

3. **프로토콜 네고시에이션 고려사항**

- 두 프로토콜을 동시에 지원하는 전환 기간 필요
- 피어 호환성 확인 필요

성능 벤치마크

처리량 비교 (예상 값)

| 메트릭 | Yamux | Mplex |

|--------|-------|-------|

| 단일 스트림 처리량 | 높음 | 중간 |

| 다중 스트림 처리량 | 매우 높음 | 중간 |

| 메모리 사용량 | 중간 | 낮음 |

| CPU 사용량 | 중간 | 낮음 |

| 지연시간 | 중간 | 낮음 |

확장성 비교

| 시나리오 | Yamux | Mplex |

|----------|-------|-------|

| 동시 스트림 수 | 1000+ | 100-500 |

| 연결 지속 시간 | 장기 | 단기-중기 |

| 네트워크 조건 | 불안정 환경 우수 | 안정 환경 적합 |

결론 및 권장사항

선택 기준

1. **Yamux 선택 시기**

- 고성능이 필요한 애플리케이션
- 복잡한 흐름 제어가 필요한 경우
- 장기 실행 연결
- 대용량 데이터 전송

2. **Mplex 선택 시기**

- 단순하고 경량한 애플리케이션
- 지연시간이 중요한 경우
- 리소스 제약 환경
- 빠른 프로토타이핑

현재 libp2p 생태계 동향

- **Yamux**: 새로운 프로젝트에서 기본 선택
- **Mplex**: 레거시 지원 및 특수 용도
- **미래 방향**: Yamux로의 점진적 마이그레이션

최종 권장사항

대부분의 새로운 libp2p 애플리케이션에서는 **Yamux**를 권장합니다. 더 나은 성능, 정교한 흐름 제어, 그리고 활발한 개발 지원을 제공하기 때문입니다. Mplex는 특별한 요구사항(극도로 낮은 지연시간, 매우 제한된 리소스)이 있는 경우에만 고려하는 것이 좋습니다.